

Haplotyping as Perfect Phylogeny: Conceptual framework and Efficient Solutions

Dan Gusfield

RECOMB, pp. 166-175, 2002

Abstract

The next high-priority phase of human genomics will involve the development of a full *Haplotype Map* of the human genome. It will be used in large-scale screens of populations to associate specific haplotypes with specific complex genetic-influenced diseases. A prototype Haplotype Mapping strategy is presently being finalized by an NIH working-group. The biological key to that strategy is the surprising fact that genomic DNA can be partitioned into long blocks where genetic recombination has been rare, leading to strikingly fewer distinct haplotypes in the population than previously expected.

In this paper we explore the algorithmic implications of the key (and new realistic) "no-recombination in long blocks" observation, for the problem of inferring haplotypes in populations. We observe that the no-recombination assumption is very powerful.

Abstract (Contd.)

This assumption, along with the standard population-genetic assumption of infinite sites imposes severe combinatorial constraints on the permitted solutions to the haplotype inference problem, leading to an efficient deterministic algorithm to deduce all features of the permitted haplotype solution(s) that can be known with certainty. The technical key is to view haplotype data as disguised information about paths in an unknown tree, and the haplotype deduction problem as a problem of reconstructing the tree from that path information. This formulation allows us to exploit deep theorems and algorithms from graph and matroid theory efficiently find one permitted solution to the haplotype problem; it gives a simple test to determine if it is the unique solution; if not, we can implicitly represent the set of all permitted solutions so that each can be efficiently created.

Genotypes and Haplotypes

- At each site, each chromosome has one of two alleles (states) denoted by 0 and 1 (Motivated by SNPs)

0	1	1	1	0	0	1	1	0
1	1	0	1	0	0	1	0	0

Two Haplotypes per individual

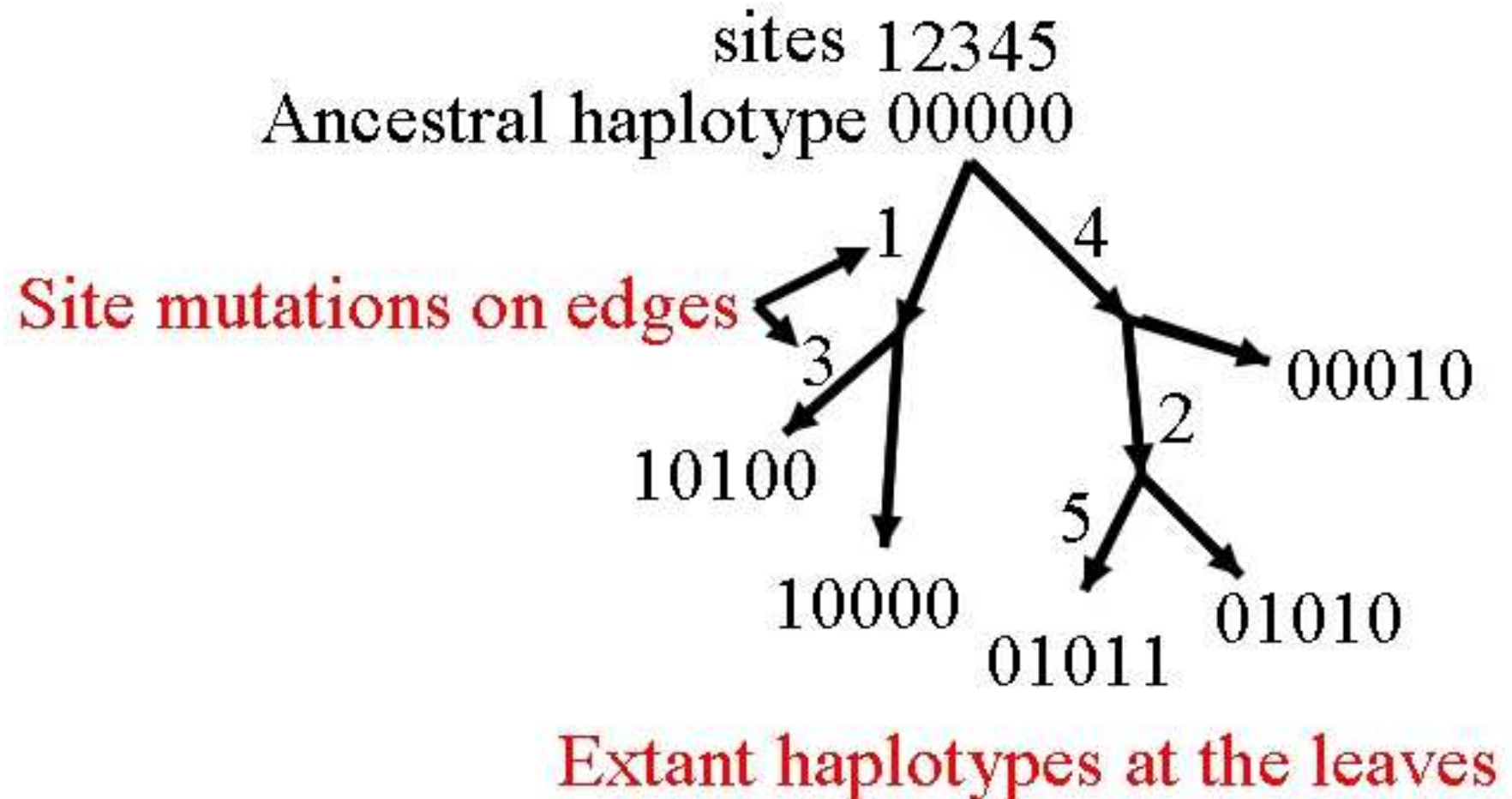
Merge the haplotypes

2 1 2 1 0 0 1 2 0

Genotype for the individual

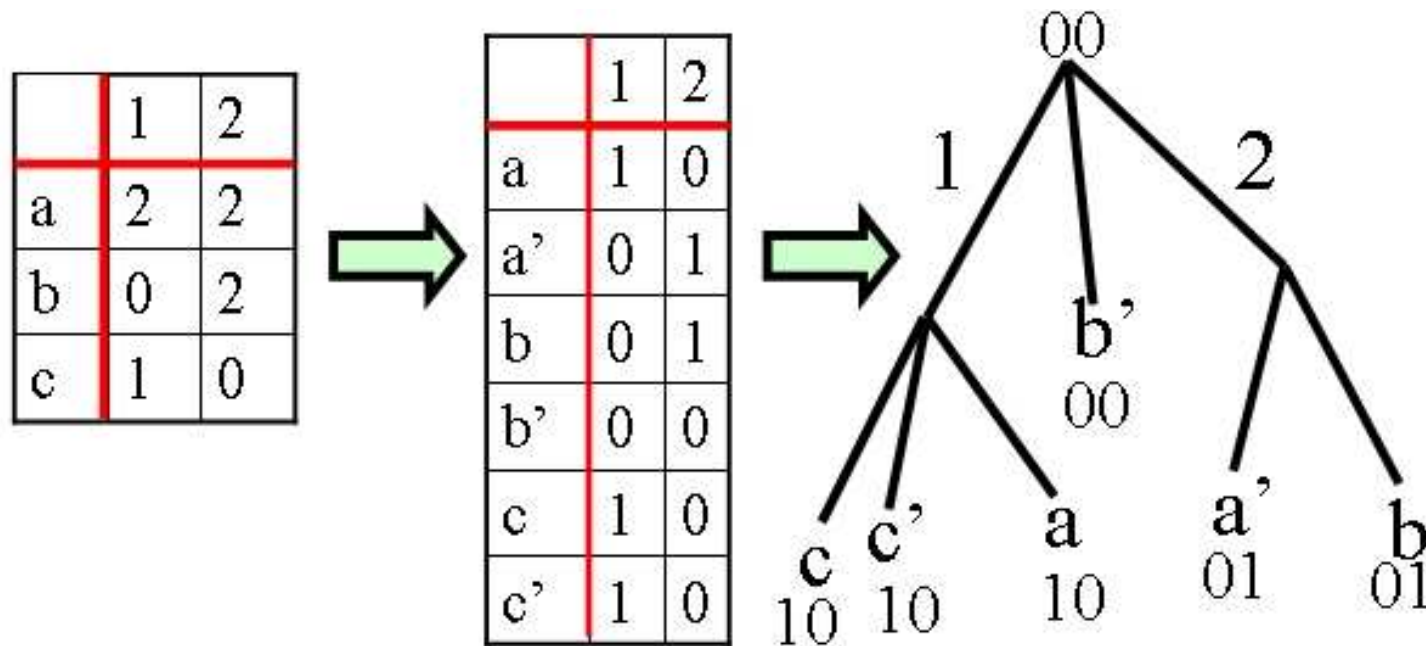
- Biological Problem:
Haplotype is more valuable than genotype data, but haplotype data is hard to collect. Genotype is easy to collect.

The Perfect Phylogeny Model



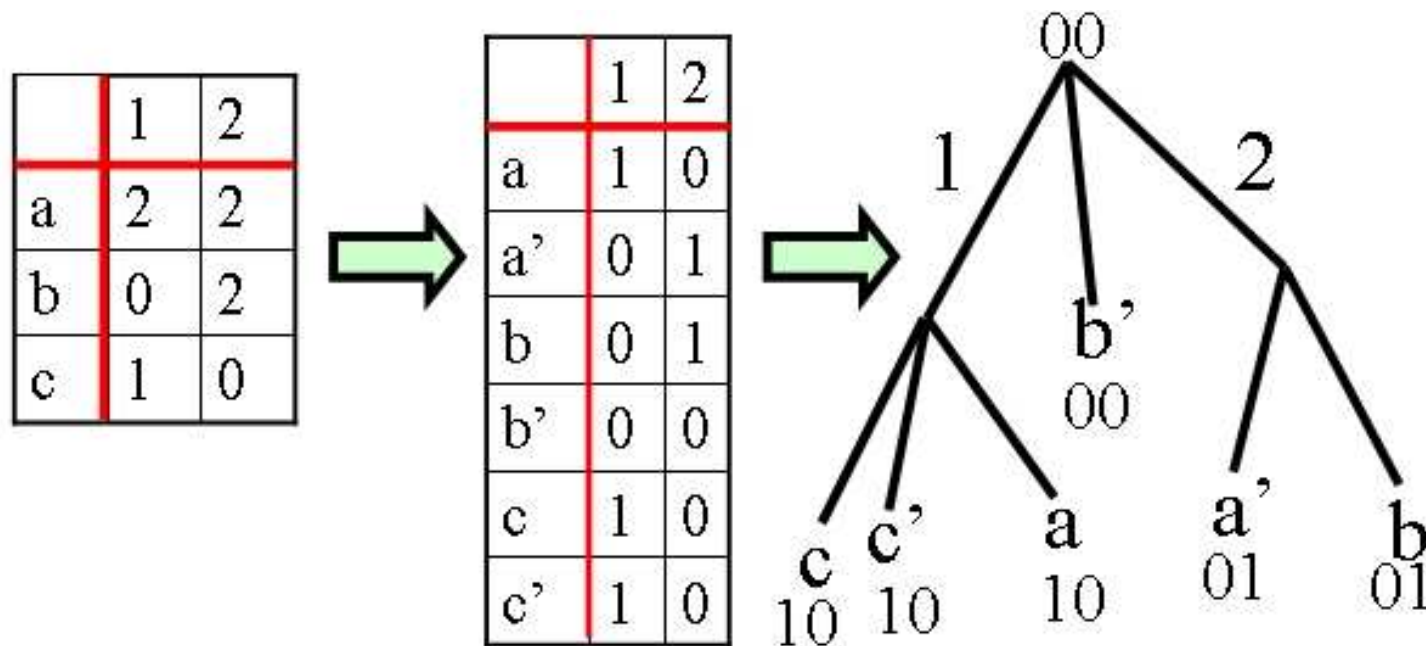
The Problem

- The Haplotype Phylogeny Problem: Given a set S of n genotype vectors, find an explaining set of haplotypes that fits a perfect phylogeny.



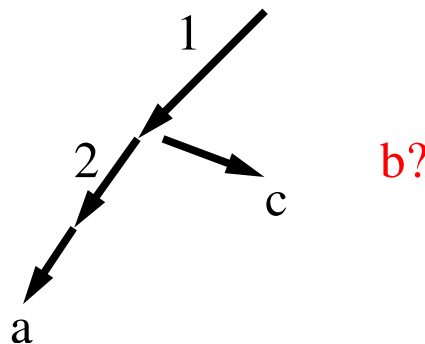
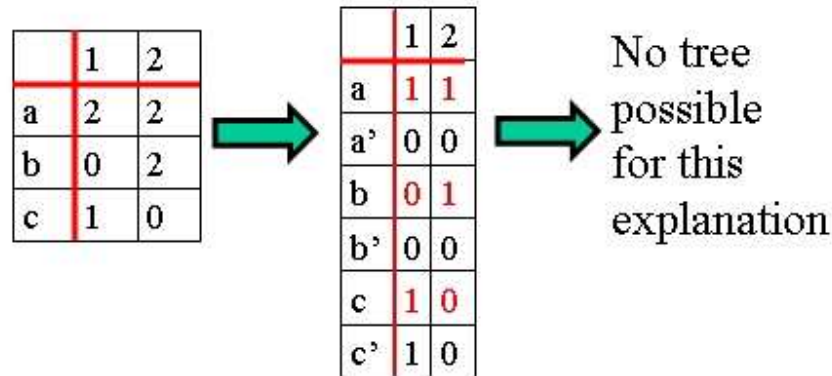
The Perfect Phylogeny

- Definition: Let $B_{2n \times m}$ be 0 – 1 matrix.
A perfect phylogeny for B is a root tree T , That obeys the proterties: (referring to paper).



The Classical Theorem of Perfect Phylogeny

- A binary matrix B has a perfect phylogeny if and only if for each pair of columns, there are no three rows with values 0, 1; 1, 0; and 1, 1 in those two columns.



Solving the PPH Problem

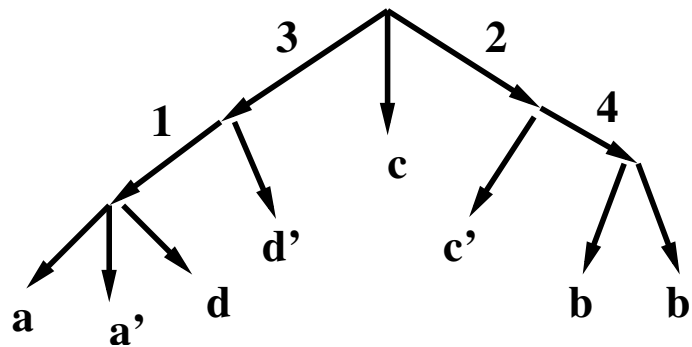
- Simple Tools based on classical Perfect Phylogeny Problem.
- Complex Tools based on Graph Realization Problem (graphic matroid realization).

Example

- Leaf Counting (LC) : In any column c , count two for each 1, and count one for each.
- Example:

	1	2	3	4
A	1	0	1	0
B	0	1	0	1
C	0	2	0	0
D	2	0	1	0
LC	3	3	4	2

	1	2	3	4
a	1	0	1	0
a'	1	0	1	0
b	0	1	0	1
b'	0	1	0	1
c	0	0	0	0
c'	0	0	0	0
d	1	0	1	0
d'	0	0	1	0



A Useful Sufficinet Condition

- Corollary: If every column of S has at least one 1, then there is a unique perfect phylogeny for S , if there is any.
- Remaining problem: 2-entries in columns without any 1-entries.
- Examples:

	1	2	3	4
A	1	0	1	0
B	0	1	0	1
C	0	2	0	0
D	2	0	1	0

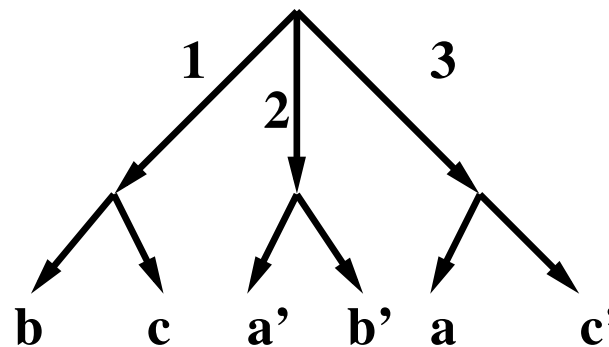
	1	2	3
A	0	2	2
B	2	2	0
C	2	0	2

Uniqueness without 1's

■ A example:

	1	2	3
A	0	2	2
B	2	2	0
C	2	0	2

	1	2	3
a	0	0	1
a'	0	1	0
b	1	0	0
b'	0	1	0
c	1	0	0
c'	0	0	1

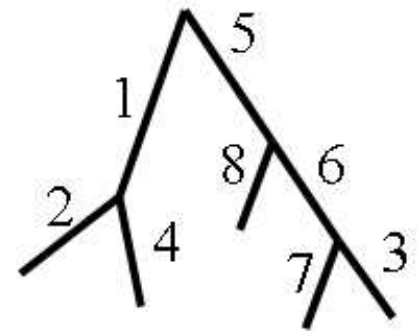


- There are 8 haplotype solutions that can explain the data, but only one that fits a perfect phylogeny.

Complex Tool

- Graph Realization: Let R_n be the integer 1 to n , and let P be an unordered subset R_n . P is called a *path set*.
input: Given E_r and a family $\Pi = P_1, P_2, \dots, P_k$ of path-sets.
output: A tree T in which each path-set is realized.
- Graph Realization Example:

P1: 1, 5, 8
P2: 2, 4
P3: 1, 2, 5, 6
P4: 3, 6, 8
P5: 1, 5, 6, 7



Realizing Tree T

Reducing PPH to Graph realization

- *Polynomial time* (almost linear-time) algorithm exist for the graph realization problem.
The algorithm are not simple; no known implementations.
- We solve any instance of the PPH problem by creating appropriate *path sets*.

Thanks

Thanks

Date: Nov. 05, 2004

Created by: Liang-Tai Wei